

Scénario d'intrusion

Injection SQL sur application web

Exploitation de DVWA, détection par Suricata et visualisation dans ELK

Projet	Projet pratique 1 Système de détection d'intrusion (IDS/IPS)
Scénario	Scénario 5 Injection SQL sur application web
Catégorie	Attaque web applicative (couche 7 - OWASP A03:2021 Injection SQL)
Cible	DVWA (Damn Vulnerable Web Application), conteneur Docker
IDS/IPS	Suricata 8.0.3 règle locale sid 1000003
Pile de supervision	Filebeat → Elasticsearch 8.19.21 → Kibana 8.19.21 (ELK)
Hôtes	Attaquant Windows 192.168.56.1 — Cible VM IDS-Lab 192.168.56.101
Date	

1. Objectif du scénario

Ce scénario vise à démontrer la détection d'une attaque web applicative de type injection SQL (SQLi). L'injection SQL consiste à insérer du code SQL malveillant dans un champ de saisie d'une application web mal protégée, afin de détourner la requête envoyée à la base de données ici pour extraire des données confidentielles (identifiants et empreintes de mots de passe).

Ce scénario complète la couverture du projet sur l'axe applicatif (couche 7 du modèle OSI, catégorie Injection du Top 10 de l'OWASP).

Résultat attendu : l'injection extrait les comptes de la base de données de l'application, une règle Suricata dédiée détecte la charge malveillante dans la requête HTTP, l'alerte est journalisée (fast.log) puis remonte dans Kibana.

2. Environnement et architecture

L'application cible DVWA est déployée dans un conteneur Docker sur la VM IDS-Lab. L'attaque est menée depuis le navigateur du poste Windows, via le réseau privé hôte (Host-Only) isolé d'Internet. Suricata, en écoute sur ce réseau, inspecte le trafic HTTP.

- **Attaquant :** poste Windows 11, 192.168.56.1, navigateur web.
- **Cible :** DVWA en conteneur Docker sur la VM IDS-Lab (192.168.56.101), port TCP/80.
- **Capteur :** Suricata 8.0.3 en écoute sur enp0s8, analyse applicative HTTP activée.
- **Supervision :** Suricata (eve.json) → Filebeat → Elasticsearch → Kibana.

3. Déploiement de la cible vulnérable (DVWA)

DVWA est une application web PHP/MySQL volontairement vulnérable, conçue pour l'apprentissage. Elle est lancée en un seul conteneur, exposé sur le port 80 de la VM.

► VM déployer et vérifier le conteneur

```
sudo docker run -d --name dvwa -p 80:80 vulnerables/web-dvwa
sudo docker ps          # vérifier : dvwa Up 0.0.0.0:80->80/tcp
```

Depuis le navigateur Windows, l'application est accessible à l'adresse **http://192.168.56.101/**. Préparation avant l'attaque : connexion avec le compte admin / password, création de la base via « Setup / Reset DB » → « Create / Reset Database », puis réglage du niveau de sécurité sur « Low » (menu DVWA Security) afin que l'injection ne soit pas filtrée.

4. Règle de détection Suricata

Une règle locale personnalisée (sid 1000003) est ajoutée pour détecter la charge d'injection. Elle inspecte l'URI de la requête HTTP et déclenche une alerte lorsque les mots-clés « union » et « select » y apparaissent signature d'une injection SQL de type UNION.

Pourquoi une règle locale plutôt que les règles Emerging Threats : les règles ET « WEB_SERVER SQL Injection » sont écrites pour du trafic \$EXTERNAL_NET → \$HOME_NET (attaque venant de l'extérieur). Or l'attaquant (192.168.56.1) appartient au même réseau que la cible, donc à HOME_NET : il n'est pas « externe » et ces règles ne se déclenchent pas. Une règle locale dédiée garantit la détection et reste cohérente avec les autres scénarios du projet.

► VM ajouter la règle dans le fichier réellement chargé, puis recharger

Rappel : le moteur lit /var/lib/suricata/rules/local.rules (chemin indiqué par default-rule-path dans suricata.yaml), et non /etc/suricata/rules/.

```
echo 'alert http any any -> $HOME_NET any (msg:"LOCAL SQL Injection (UNION SELECT) detectee";
flow:to_server; http.uri; content:"union"; nocase; content:"select"; nocase; distance:0; classtype:web-application-
attack; sid:1000003; rev:1;)' | sudo tee -a /var/lib/suricata/rules/local.rules
```

```
sudo suricata -c reload-rules    # réponse : {"message":"done","return":"OK"}
```

5. L'attaque : injection SQL de type UNION

Une fois connecte a la page DVWA avec les identifiant ad,in et password ; Sur la page « SQL Injection » de DVWA (niveau Low), la requette suivante est saisie dans le champ User ID puis soumise :

```
1' UNION SELECT user,password FROM users-- -
```

Cette requête ferme la valeur attendue (1'), ajoute une clause UNION SELECT qui récupère les colonnes user et password de la table users, et neutralise la fin de la requête d'origine (-- -, commentaire SQL). L'application affiche alors l'ensemble des comptes avec leurs empreintes de mots de passe, preuve que l'injection a aboutie.

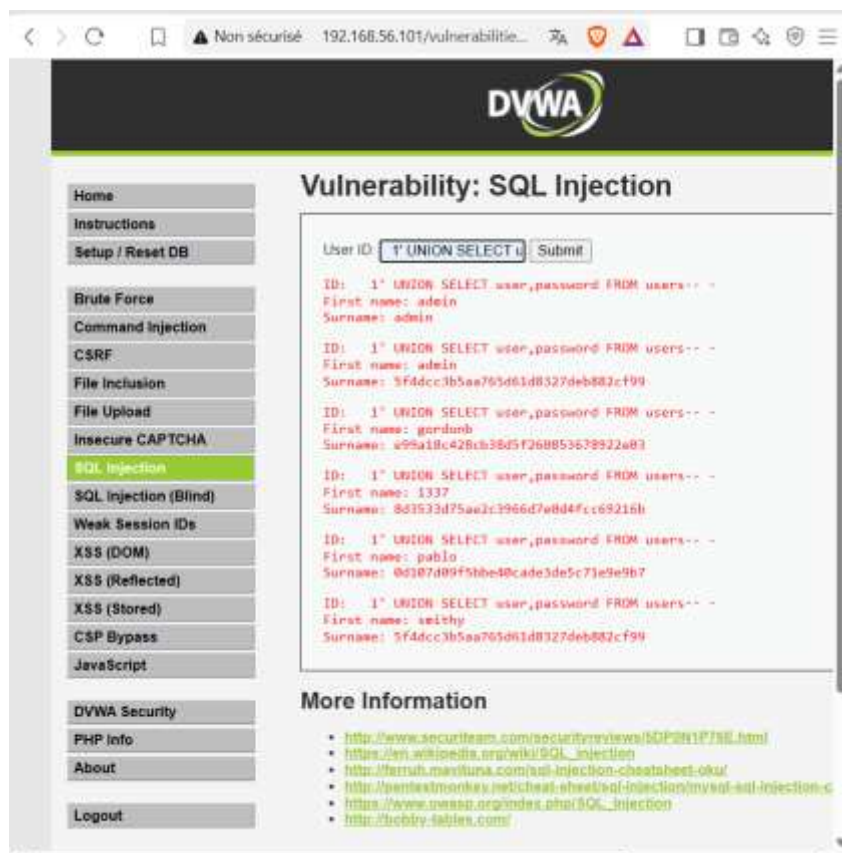


Figure 1 - Résultat de l'injection dans DVWA : la requête détournée affiche tous les comptes et leurs empreintes MD5.

Observation : les empreintes extraites sont des MD5 non salés. Par exemple 5f4dcc3b5aa765d61d8327deb882cf99 correspond au mot « password » d'où deux comptes partageant la même empreinte. Cela illustre une mauvaise pratique de stockage de mots de passe, mise en évidence par l'attaque.

6. Détection par Suricata (fast.log)

Lors de la soumission, la charge transite dans l'URL de la requête HTTP (méthode GET). Suricata inspecte l'URI, y repère la séquence « union ... select » et déclenche la règle 1000003, journalisée dans fast.log avec la classification « Web Application Attack » en priorité 1.

► VM vérifier la détection

```
sudo grep -a 1000003 /var/log/suricata/fast.log | tail -5
```

```

madeo@IDS-Lab:~$ echo 'alert http any any -> $HOME_NET any (msg:"LOCAL SQL Injection (UNION SELECT) detectee"; flow:to_server; http.uri; content:"union"; nocase; content:"select"; nocase; distance:0; classtype:web-application-attack; sid:1000003; rev:1;)' | sudo tee -a /var/lib/suricata/rules
madeo@IDS-Lab:~$ sudo suricata -c reload-rules
{"message": "done", "return": "OK"}
madeo@IDS-Lab:~$ sudo grep -a 1000003 /var/log/suricata/fast.log | tail -5
10/04/2026-14:43:43.475078  [**] [1:1000003:1] LOCAL SQL Injection (UNION SELECT) detectee [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.56.1:25588 -> 192.168.56.101:80
madeo@IDS-Lab:~$ |

```

Figure 2 Ajout de la règle, rechargement et alerte « LOCAL SQL Injection (UNION SELECT) detectee » (sid 1000003) dans fast.log.

7. Visualisation dans Kibana

L'alerte remonte automatiquement dans **Elasticsearch** via **Filebeat**. Dans l'onglet Discover (vue de données filebeat-*), le filtre KQL suivant isole l'événement d'injection :

► Kibana requête KQL (onglet Discover, vue filebeat-*)

```
event.dataset: "suricataeve" and rule.id: "1000003"
```

La capture correspondant confirme la détection : rule.id 1000003, destination.ip 192.168.56.101, destination.port 80, horodatage concordant avec l'attaque. L'intérêt de ce scénario est que l'événement contient la charge d'attaque en clair dans les champs d'URL, offrant une preuve directement exploitable.

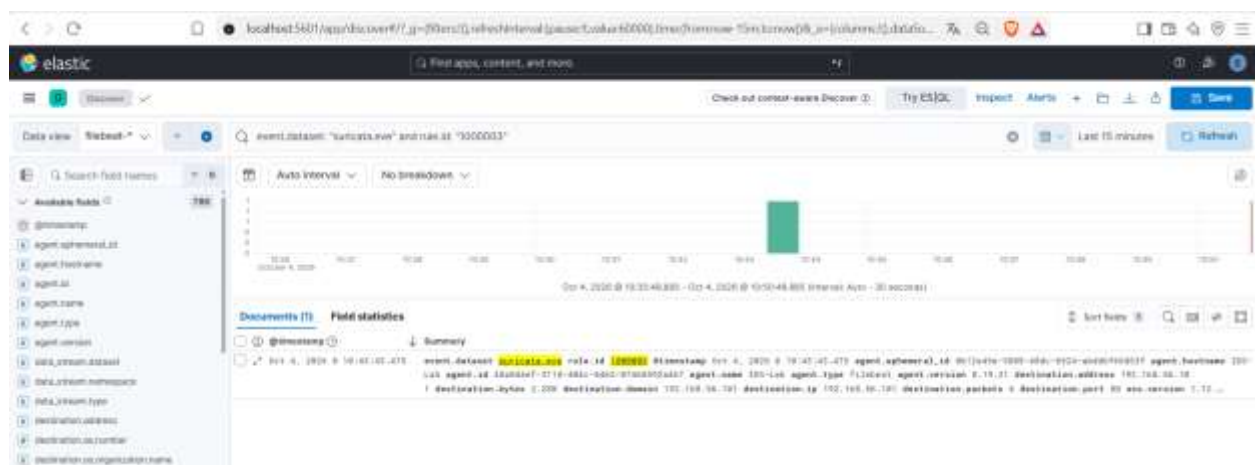


Figure 3 — Kibana (Discover, filebeat-*) : l'alerte d'injection SQL sid 1000003 remontée dans la pile ELK.

8. Conclusion

Le scénario démontre la chaîne complète de détection d'une attaque web applicative : déploiement d'une cible vulnérable, exploitation réelle d'une injection SQL aboutissant à l'extraction de données, détection par une règle Suricata dédiée, journalisation locale puis remontée et visualisation dans ELK. Il ajoute au projet la dimension applicative (OWASP, couche 7).